

Deep Convolutional Neural Networks

Vajira Thambawita

Ref: Raschka, Sebastian; Liu, Yuxi (Hayden); Mirjalili, Vahid. Machine Learning with PyTorch and Scikit-Learn: Develop machine learning and deep learning models with Python.

Outline

What is a CNN?

CNN Blocks?

Implementing CNN models using Pytorch

Identifying differences in Classification Detection, and Segmentation

What are CNNs?

- CNNs are inspired by how the visual cortex of the human brain works when recognizing objects.

The human visual cortex



The original discovery of how the visual cortex of our brain functions was made by David H. Hubel and Torsten Wiesel in 1959, when they inserted a microelectrode into the primary visual cortex of an anesthetized cat. They observed that neurons respond differently after projecting different patterns of light in front of the cat. This eventually led to the discovery of the different layers of the visual cortex. While the primary layer mainly detects edges and straight lines, higher-order layers focus more on extracting complex shapes and patterns.

Some key words

Salient features: relevant features, which is key to the performance of any machine learning algorithm. For traditional machine learning models, we extract features from domain experts or based on computational feature extraction.

Deep NNs, such as CNNs automatically learn features from raw data.

In this regard, we consider CNN layers are feature extractors.

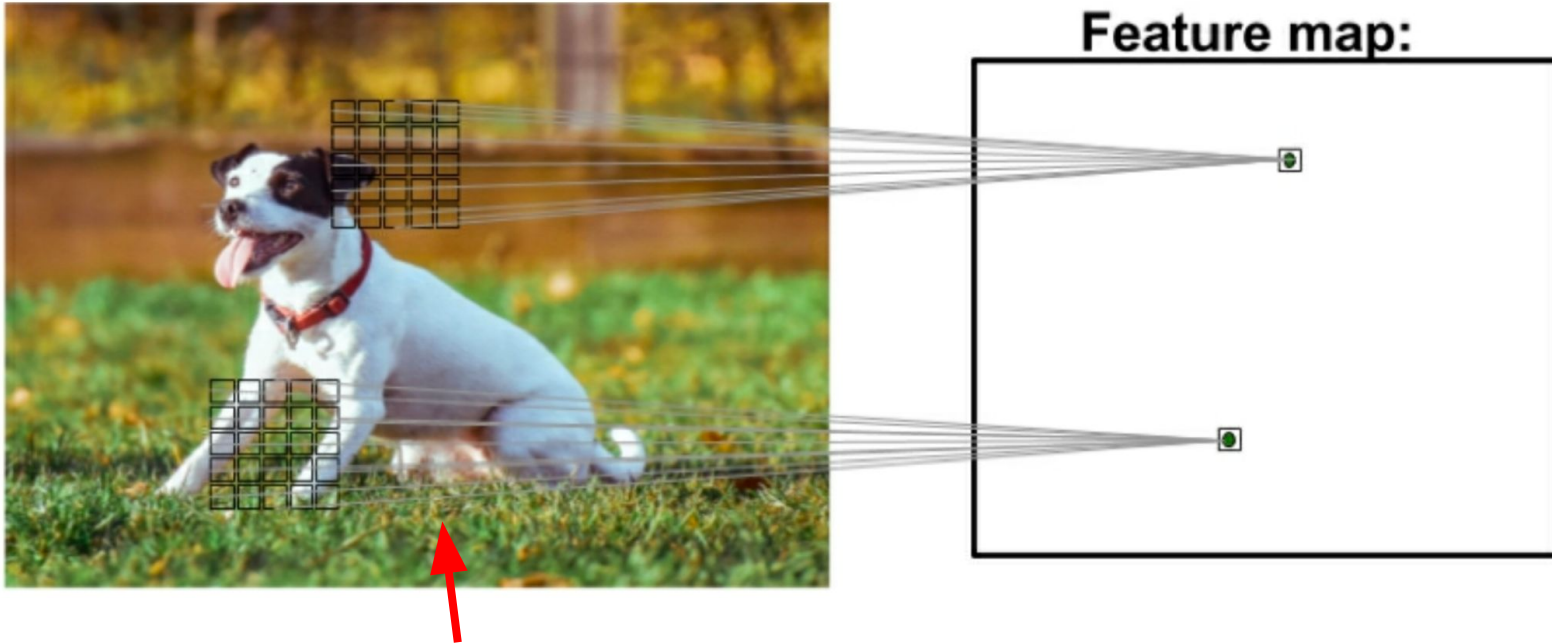
- Early layers: extract low level features
- Later layers (fully connected layers or MLP): use extracted features to predict the output.

CNNs have feature hierarchy: from low-level features to high-level features. (Ex: low-level features: edges, blobs, high-level features: general contours of object, like cats, dogs)

Feature maps

In machine learning, especially in convolutional neural networks (CNNs), the local receptive field refers to the small region of input data (like pixels in an image) that a single neuron (or unit) in a layer processes. Instead of processing an entire image at once, a CNN looks at small patches (local receptive fields) of the image and applies filters or convolutions to detect patterns such as edges, textures, or colors. These small patches of input are then combined to form more complex patterns in deeper layers.

CNN compute feature maps



A local patch of pixels : **local receptive field**

Two important ideas of CNNs

1. Sparse connectivity:

- A single element of a feature map is connected to only a small patch of pixels. (compare with MLP)
- In images, local pixels are more likely to be related to each other than distant pixels. Sparse connectivity allows the network to focus on these local relationships, which reduces the number of parameters and computational cost, making the network more efficient.

2. Parameter sharing

- The same weights are used for different patches of the input image.
- In images, the same features (like edges, corners, textures) can appear in different locations. Instead of learning a separate filter for each location, CNNs use a single filter across the entire image. This drastically reduces the number of parameters to learn and ensures that the network can detect a feature regardless of where it appears in the image.

As a result, convolutional layers substantially decrease the number of weights (parameters) in the network. (compared to MLP)

Typical CNN

Consist of

- Convolutional and subsampling layers + one or two full connected layers

Subsampling layers (pooling layers):do not have any learnable parameters.

But, convolutional and fully connected layers have learnable parameters: weight and bias.

Discrete convolution (convolution) : in one dimension

Some notations:

A discrete convolution of two vectors, x and w :

$$y = x * w$$

(note that $*$ is not multiplication in Python, it is convolution operation)

Input (or signal)

filter(or kernel)

Convolution operation

Discrete convolution (mathematical formula)

$$\mathbf{y} = \mathbf{x} * \mathbf{w} \rightarrow y[i] = \sum_{k=-\infty}^{+\infty} x[i - k] w[k]$$

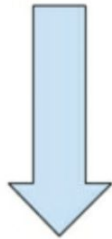
i = iterate through the each element of output vector y .

$-\infty$ to $+\infty$? if x has 10 features, then indices are 0,1,2,...,9, then $-\infty = -1$ and $+\infty = 10$, so we need zero-padding (padding (p)).

Padding

Original $\mathbf{x}$:

3	2	1	7	1	2	5	4
---	---	---	---	---	---	---	---



Padding
with $p=2$

0	0	3	2	1	7	1	2	5	4	0	0
---	---	---	---	---	---	---	---	---	---	---	---

Updated formula for discrete convolution

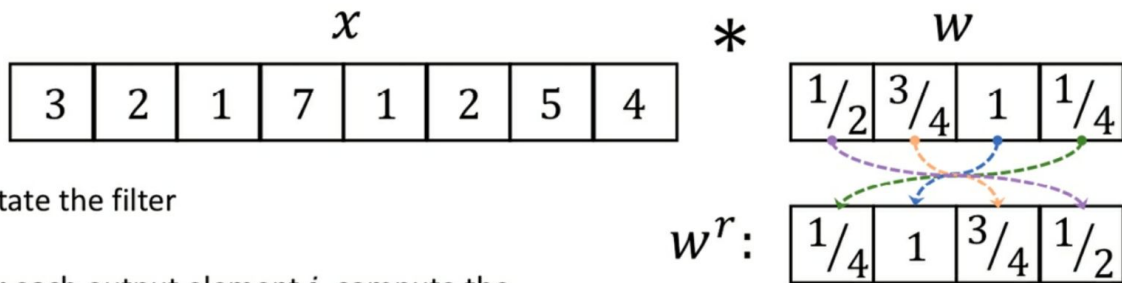
$$\mathbf{y} = \mathbf{x} * \mathbf{w} \rightarrow y[i] = \sum_{k=0}^{m-1} x^p[i + m - k] w[k]$$

$\mathbf{x}$ has n elements

$\mathbf{w}$ has m elements ($m < n$)

Padded vector $\mathbf{x}^p$ has size $n+2p$

Example



Step 1: Rotate the filter

Step 2: For each output element i , compute the dot-product $x[i:i+4] \cdot w^r$

(move the filter two cells)

Stride = 2 (shift)

$$y[0] = 3 \times \frac{1}{4} + 2 \times 1 + 1 \times \frac{3}{4} + 7 \times \frac{1}{2}$$

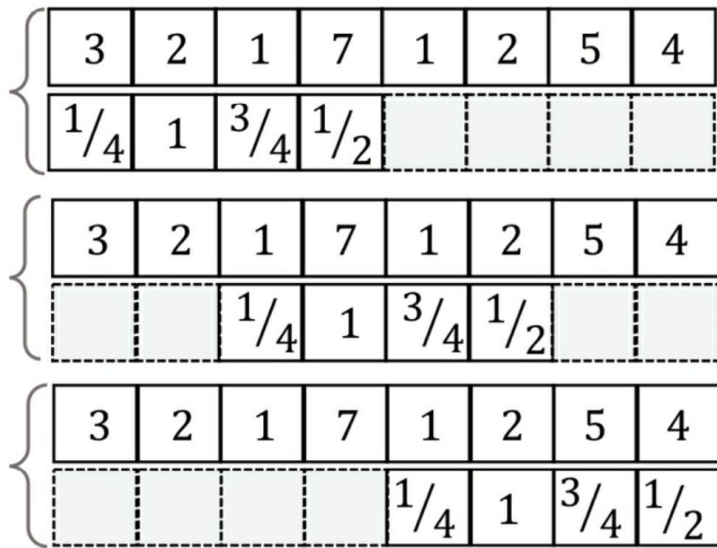
$$\rightarrow y[0] = 7$$

$$y[1] = 1 \times \frac{1}{4} + 7 \times 1 + 1 \times \frac{3}{4} + 2 \times \frac{1}{2}$$

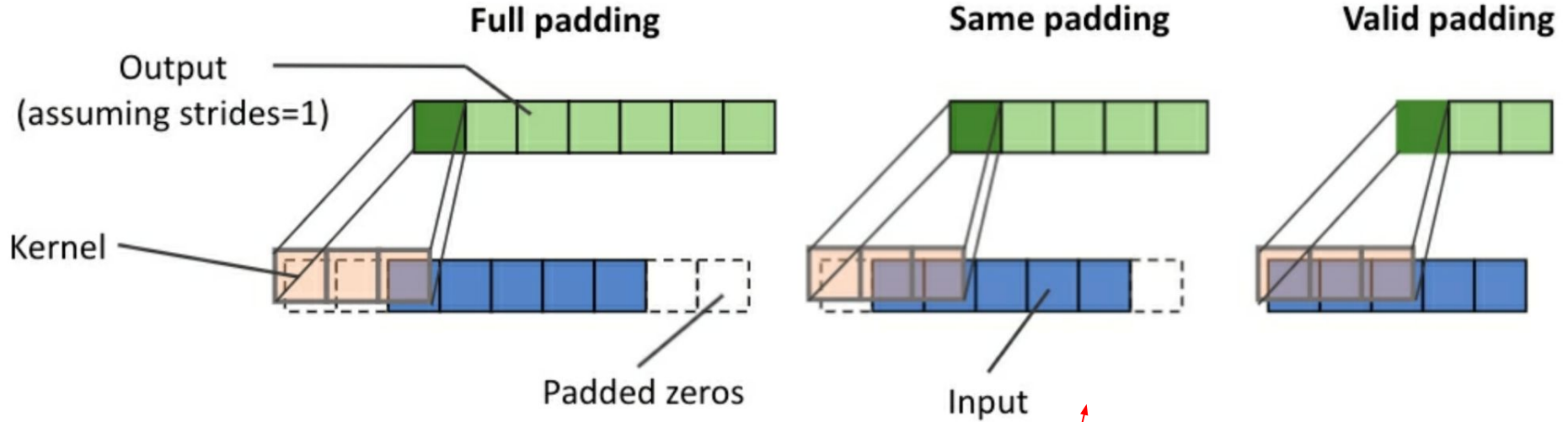
$$\rightarrow y[1] = 9$$

$$y[2] = 1 \times \frac{1}{4} + 2 \times 1 + 5 \times \frac{3}{4} + 4 \times \frac{1}{2}$$

$$\rightarrow y[2] = 8$$



Padding



Most common in CNNs

The size of the convolution output

$$o = \left\lfloor \frac{n + 2p - m}{s} \right\rfloor + 1$$

Input size

Padding size

Kernel size

Stride size

Example

- Compute the output size for an input vector of size 10 with a convolution kernel of size 5, padding 2, and stride 1:

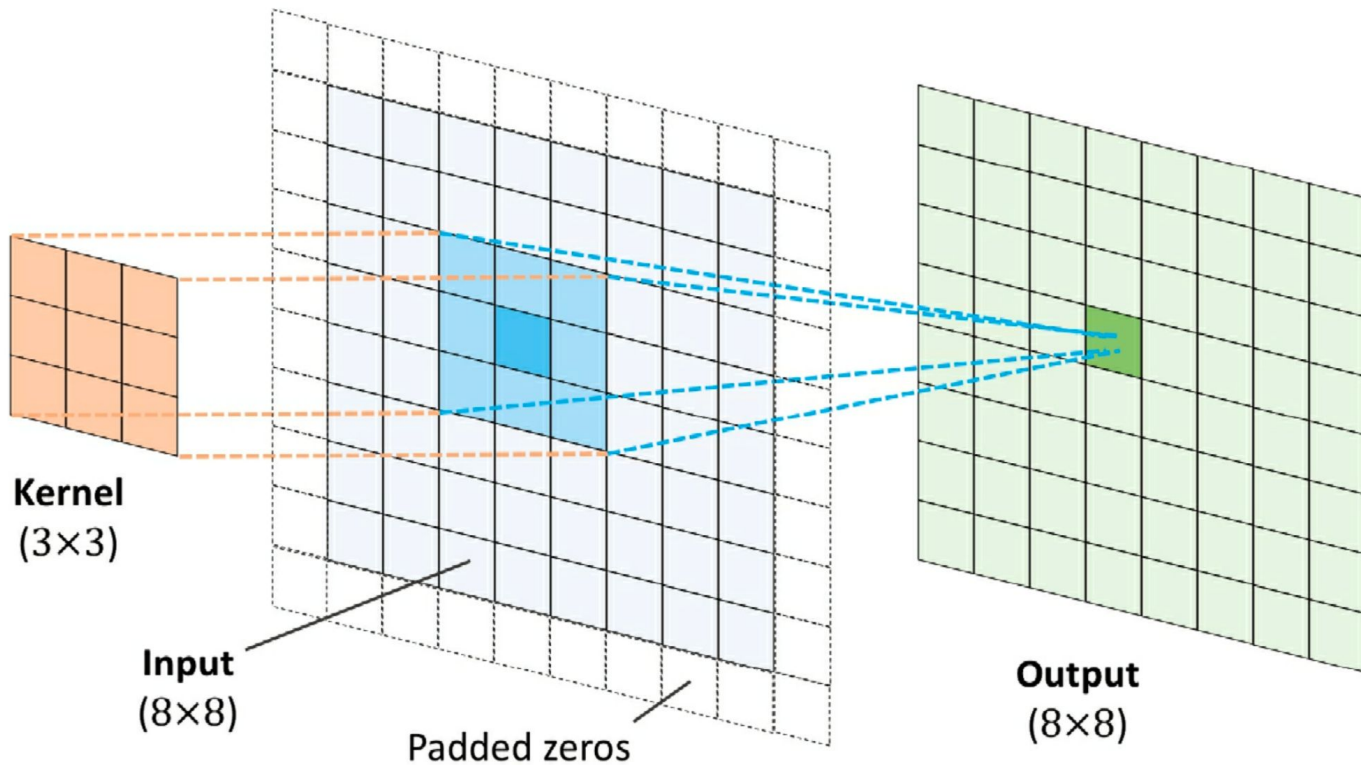
$$n = 10, m = 5, \quad p = 2, \quad s = 1 \rightarrow o = \left\lceil \frac{10 + 2 \times 2 - 5}{1} \right\rceil + 1 = 10$$

- How does the output size change for the same input vector when we have a kernel of size 3 and stride 2?

$$n = 10, m = 3, \quad p = 2, \quad s = 2 \rightarrow o = \left\lceil \frac{10 + 2 \times 2 - 3}{2} \right\rceil + 1 = 6$$

2D Convolution

$$Y = X * W \rightarrow Y[i, j] = \sum_{k_1=-\infty}^{+\infty} \sum_{k_2=-\infty}^{+\infty} X[i - k_1, j - k_2] W[k_1, k_2]$$



Example

X

0	0	0	0	0
0	2	1	2	0
0	5	0	1	0
0	1	7	3	0
0	0	0	0	0

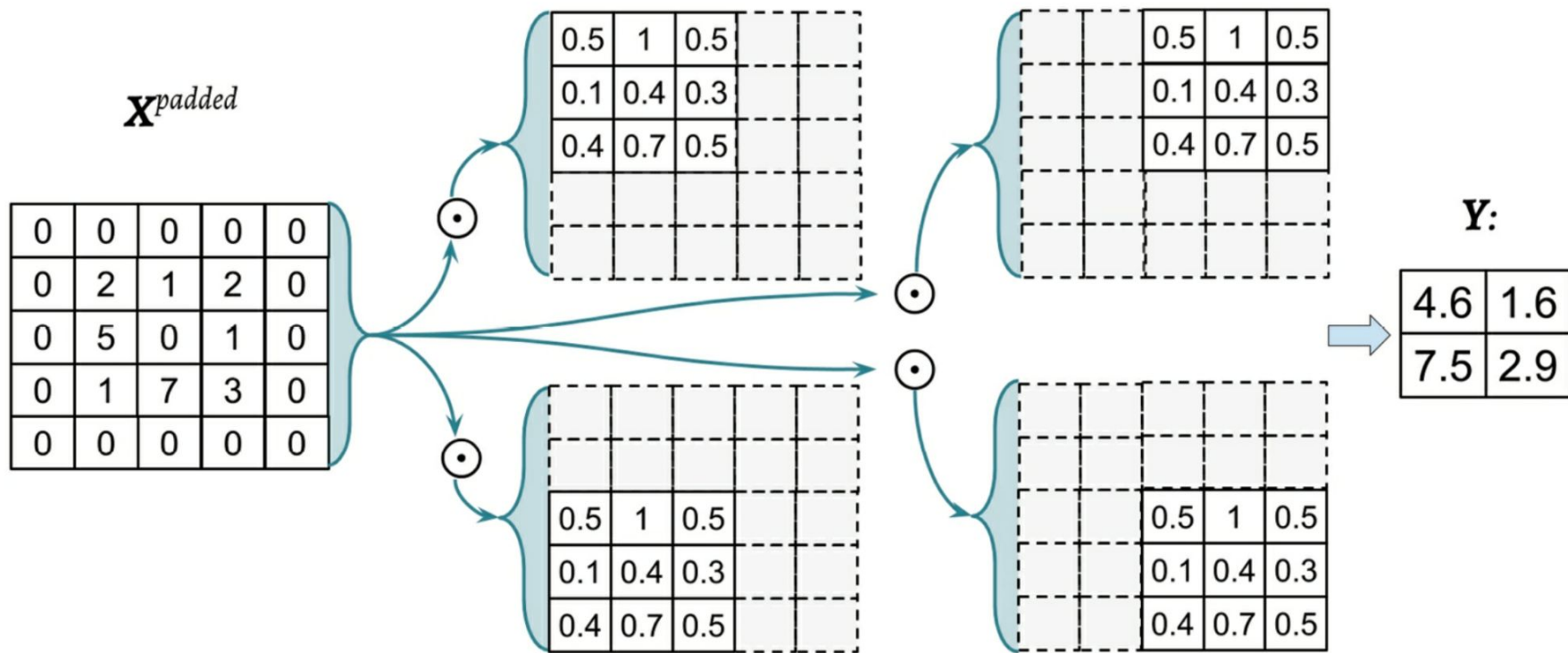
*

W

0.5	0.7	0.4
0.3	0.4	0.1
0.5	1	0.5

Rotated filter

$$\mathbf{W}^r = \begin{bmatrix} 0.5 & 1 & 0.5 \\ 0.1 & 0.4 & 0.3 \\ 0.4 & 0.7 & 0.5 \end{bmatrix}$$



Subsampling layers

Pooling ($P_{3 \times 3}$)

2	1	7	1	2	5
5	0	3	4	1	2
1	7	8	3	3	0
0	3	2	0	1	1
6	2	5	3	0	3
3	6	0	2	1	0

Max-pooling

8	5
6	3

Mean-pooling

3.78	2.33
3	1.22

Note:
stride=(3, 3)

Advantages of Pooling

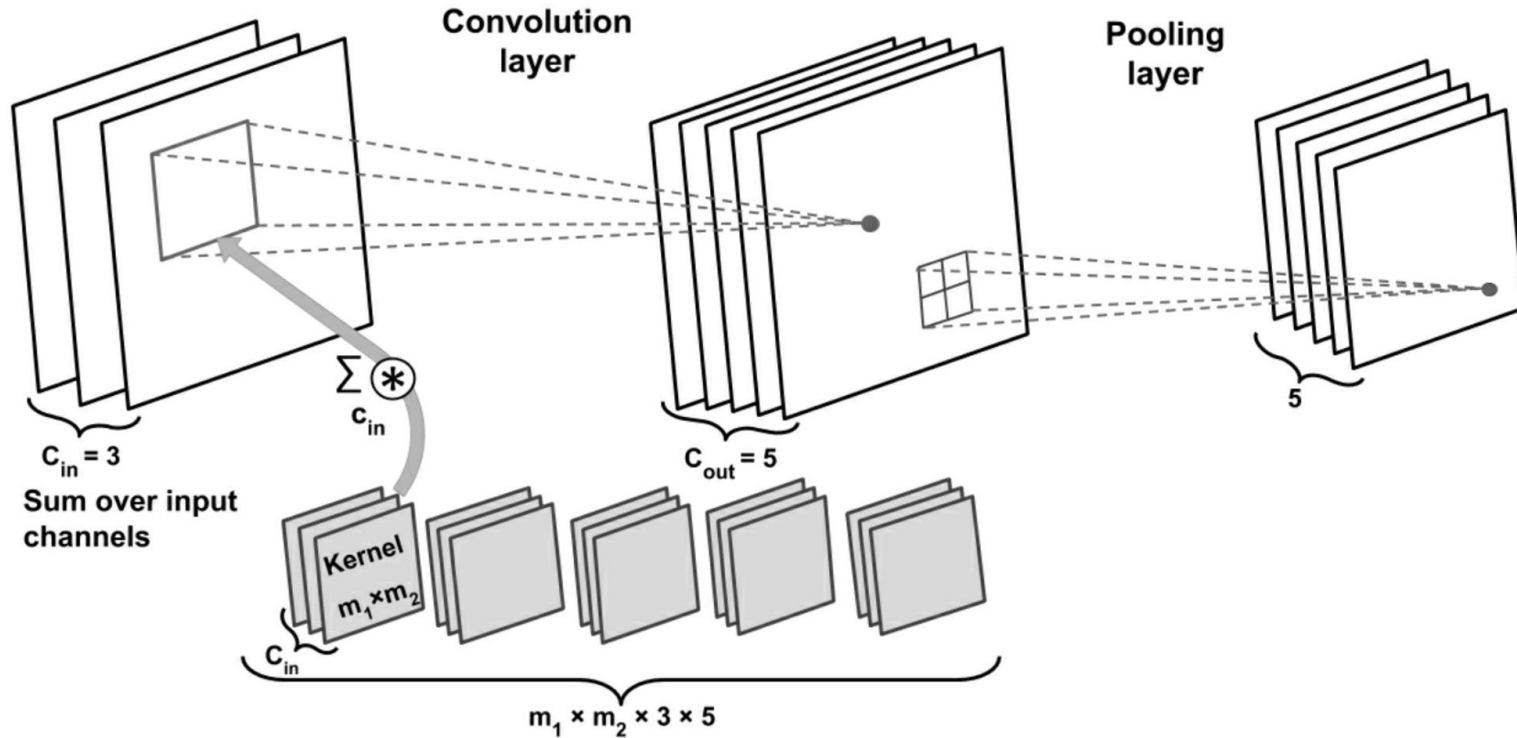
1. Pooling (max-pooling) introduce local invariance: the small changes in local neighborhood do not change the result of max pooling.
2. Pooling decreases the size of features

$$\begin{array}{l}
 \mathbf{X}_1 = \begin{bmatrix} 10 & 255 & 125 & 0 & 170 & 100 \\ 70 & 255 & 105 & 25 & 25 & 70 \\ 255 & 0 & 150 & 0 & 10 & 10 \\ 0 & 255 & 10 & 10 & 150 & 20 \\ 70 & 15 & 200 & 100 & 95 & 0 \\ 35 & 25 & 100 & 20 & 0 & 60 \end{bmatrix} \\
 \mathbf{X}_2 = \begin{bmatrix} 100 & 100 & 100 & 50 & 100 & 50 \\ 95 & 255 & 100 & 125 & 125 & 170 \\ 80 & 40 & 10 & 10 & 125 & 150 \\ 255 & 30 & 150 & 20 & 120 & 125 \\ 30 & 30 & 150 & 100 & 70 & 70 \\ 70 & 30 & 100 & 200 & 70 & 95 \end{bmatrix}
 \end{array}
 \left. \vphantom{\begin{array}{l} \mathbf{X}_1 \\ \mathbf{X}_2 \end{array}} \right\} \xrightarrow{\text{max pooling } P_{2 \times 2}} \begin{bmatrix} 255 & 125 & 170 \\ 255 & 150 & 150 \\ 70 & 200 & 95 \end{bmatrix}$$

Another option to replace pooling layers

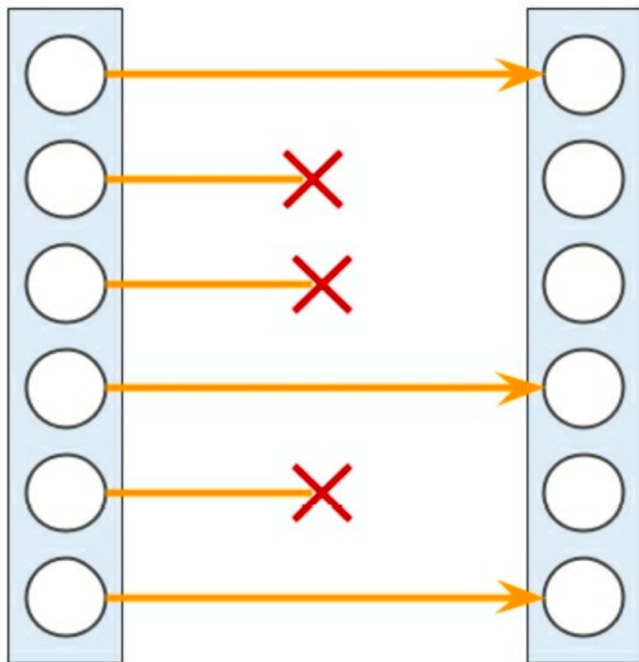
For ex: a convolutional layers with stride 2 (but this one has learnable parameters compared to pooling layers.)

All together

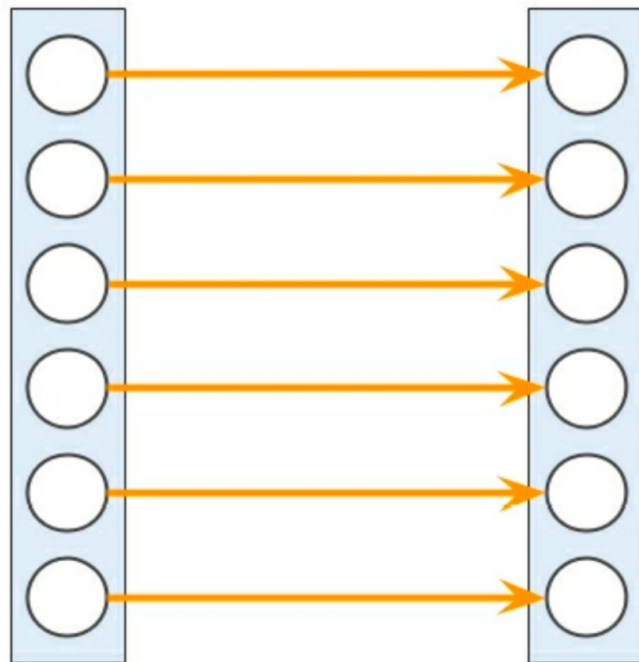


Dropout

Training:
dropout probability $p=50\%$



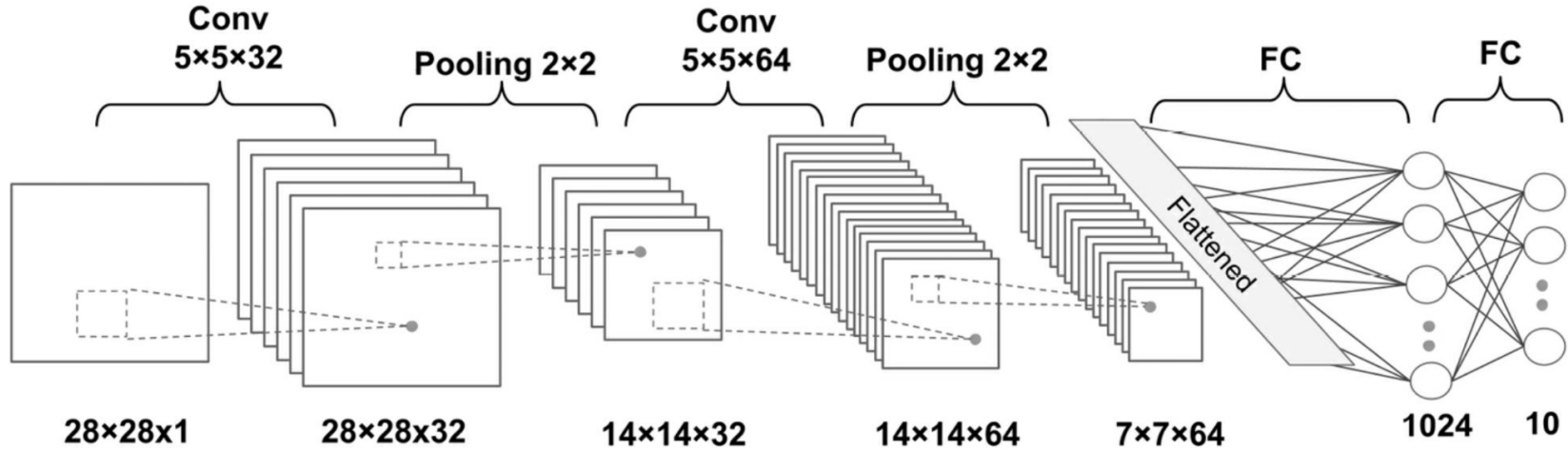
Evaluation:
use all units



Loss functions for classification

Loss function	Usage	Example <i>Using probabilities</i>	Example <i>Using logits</i>
<i>BCELoss or BCEWithLogitsLoss</i>	Binary classification	<i>BCELoss</i> $y_true:$ 1 $y_pred:$ 0.8	<i>BCEWithLogitsLoss</i> $y_true:$ 1 $y_pred:$ 0.8
<i>NLLLoss or CrossEntropyLoss</i>	Multiclass classification	<i>NLLLoss</i> $y_true:$ 2 $y_pred:$ 0.30 0.15 0.55	<i>CrossEntropyLoss</i> $y_true:$ 2 $y_pred:$ 1.5 0.8 2.1

Ex: Multilayer-CNN architecture

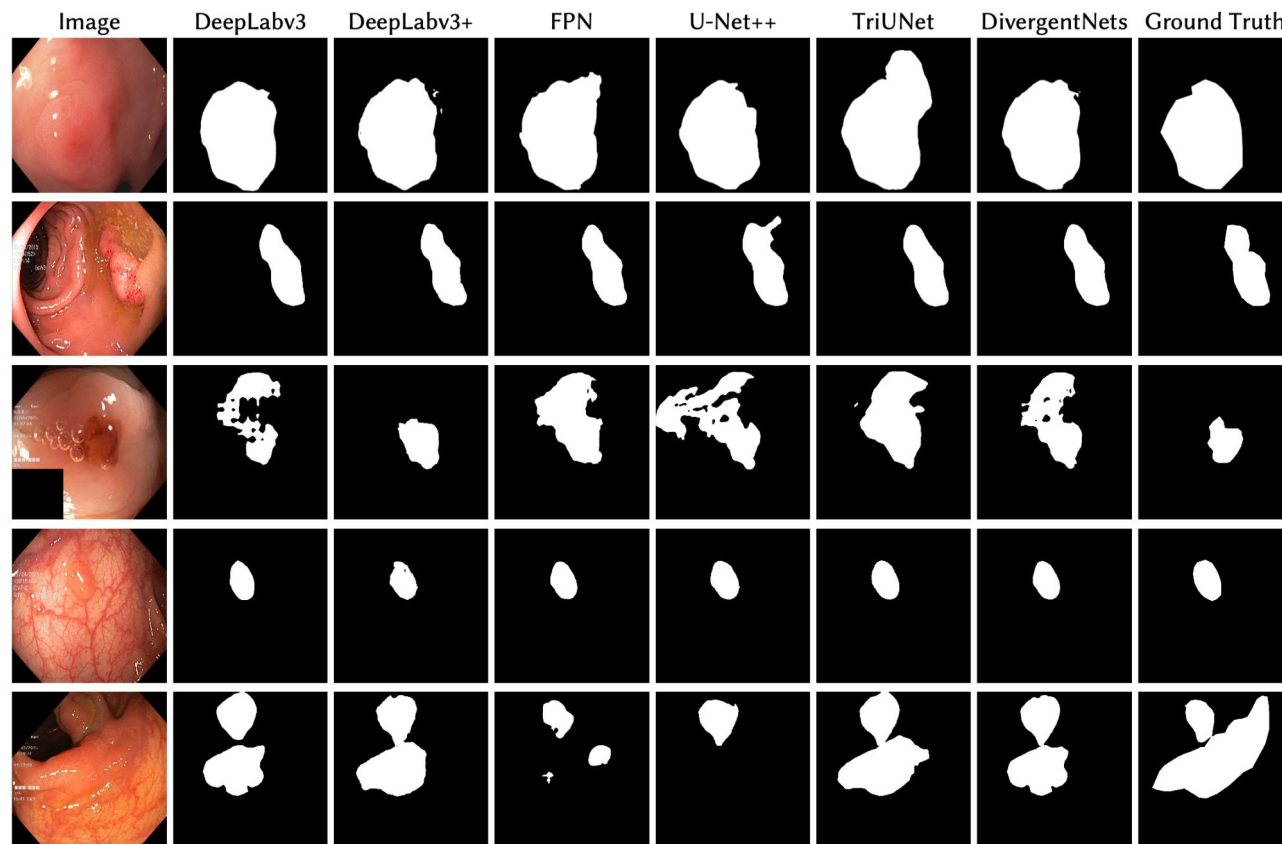


Solving a real problem (team work)

Resources

- Paper: <https://arxiv.org/abs/1505.04597>
- Code: <https://github.com/milesial/Pytorch-UNet>
- Data: <https://datasets.simula.no/kvasir-seg/>

Image segmentation



An image segmentation model: UNet

